Rapport Application Phase 3:

L'impact de l'IA sur Qualité du logiciel et de la Maintenance

Réalisé par:

-Rafi Bettaieb

IUT de Villetaneuse — Université Sorbonne Paris Nord
Année universitaire 2025–2026

Lien Github: github.com/Rafi-Bettaieb/SAE_Voronoi_E

1. Introduction

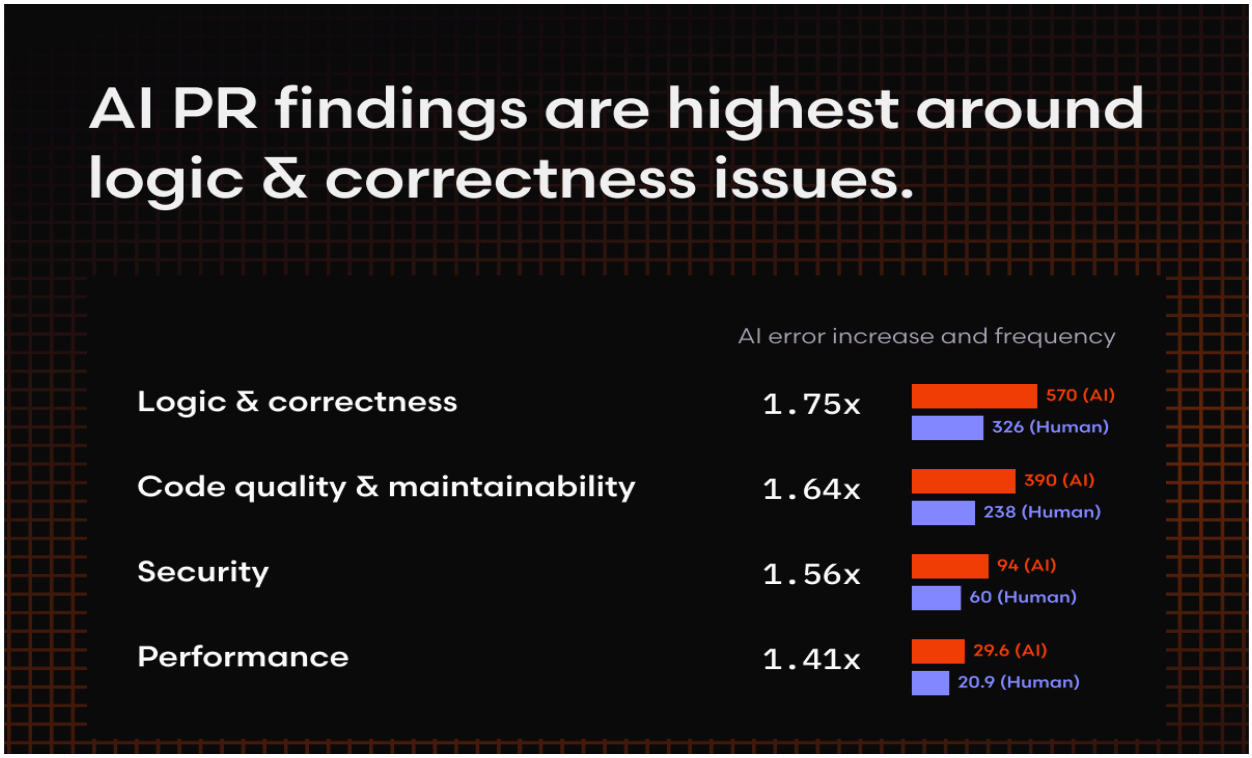
De nos jours, l'IA générative est de plus en plus utilisée dans le développement de logiciels. Ils permettent d'écrire du code plus rapidement.Ils automatisent certaines tâches et nous permettent de gagner du temps lors du développement. Cependant, l'application ne doit pas seulement fonctionner, elle doit aussi être robuste, maintenable, sécurisée et performante sur le long terme. Peut-on donc dire que l'usage de l'IA améliore la qualité du logiciel ou au contraire engendre de nouveaux problèmes ? C'est ce dont je vais vous parler dans ce rapport.

2. Impact sur la qualité du logiciel

La qualité logicielle selon Alain April et Claude Laporte est une appréciation globale d'un logiciel, basée sur de nombreux critères :

- Capacité fonctionnelle : aptitude à satisfaire correctement les besoins des utilisateurs.
- Utilisabilité : simplicité d'apprentissage et de manœuvre.
- Fiabilité : capacité à fournir des résultats corrects en toutes circonstances.
- Performance : efficacité dans l'utilisation des ressources
- Maintenabilité : facilité de correction et d'évolution du logiciel
- Portabilité : aptitude à fonctionner dans des environnements variés

Beaucoup de développeurs génèrent du code en utilisant l'IA générative et se limitent à vérifier si le code fonctionne ou pas. Si le code ne compte pas d'erreurs de compilation et renvoie le résultat correctement, il est validé mais en réalité il crée une dette technique. Autrement dit, le code est difficile à lire, mal structuré et difficile à faire évoluer. Certes, il fonctionne, mais pas dans la durée donc il n'est pas du tout maintenable.
De nos jours, les développeurs font plus de copier-coller sans même vérifier le contenu généré par l'IA s'il marchait c'est bien sinon ils font copier-coller de l'erreur et demandent à l'IA de la corriger sans essayer de comprendre quoi l'erreur ou de la résoudre eux-même. Souvent, ils ne lisent pas toutes les lignes. Ils ne comprennent pas le code à 100 % et ne prennent pas le temps d'améliorer le code ou sa structure, leurs but essentiel est de faire fonctionner le projet sans penser à sa maintenance.



source : CodeRabbit

Le code généré par l'IA crée des risques pour la maintenance. L'IA produit un code très complexe, le résultat est un code totalement illisible, le développeur aura du mal à le comprendre et à le modifier plus tard.
L'IA produit aussi de mauvais code avec des anti-patterns , appelé "code smells" ou mauvaise odeur du code puis sont des mauvaises pratiques de conception logicielle qui conduisent à l'apparition de défauts.
L'IA viole aussi le principe SOLID qu'on a vu dans la Maintenance Applicative.
Par exemple, l'IA génère des fonctions qui sont très longues avec beaucoup de responsabilité ce qui viole le principe de SRP (Single Responsibility Principle), le risque sur la maintenabilité est direct, un développeur va perdre des heures à nettoyer le code.
L'IA viole aussi le principe DRY(Don't Repeat Yourself). Au lieu de créer un composant réutilisable, l'IA copie et colle le même bloc un peu partout dans le projet, donc si on change une règle métier on doit changer le code à plusieurs endroits et on risque d'oublier ce qui va créer des bugs cachés dans toute l'application.
L'IA ne respecte pas aussi le principe OCP (Open/Closed Principle). Elle écrit des conditions très longues avec beaucoup de if et de else donc pour ajouter une nouvelle option on doit changer le code ancien qui marchait déjà, il risque donc de tout casser.
Pour l'entreprise, ces mauvaises pratiques coûtent beaucoup de temps et d'argent. Les

développeurs se bloquent sur un code illisible ce qui va ralentir toutes les nouvelles fonctionnalités à ajouter. Finalement l'application devient instable avec des bug cachés pour les clients ce qui va impacter l'expérience utilisateur .
Quand on demande à l'IA de nous donner une solution, elle choisit la solution la plus connue mais pas la plus rapide. Elle ne donne pas une solution optimale, en fait elle va utiliser un algorithme très lourd par exemple un algorithme avec une complexité de $O(n^3)$ au lieu de $O(n \log(n))$ comme on a vu dans la phase 2 de la SAE. En conséquence, le logiciel met beaucoup de temps à s'exécuter et consomme beaucoup de mémoire.
L'IA ne donne pas une solution organisé et bien structuré où elle donne toute la réponse dans un seul fichier et ne sait pas comment structurer le projet selon des modèles prédéfinis comme le modèle MVC ou le modèle MVVM et la on trouve qu'un seul fichier fait tout le travail sans déléguer les responsabilités.

3. Conclusion

En guise de conclusion, l'IA est un outil puissant qui sert à nous aider à créer des applications très vite mais elle crée une dette technique dangereuse. Donc on doit monter en compétence et ne concentrer pas uniquement sur l'écriture du code mais à l'architecture et la conception de haut niveau et doit toujours lire et vérifier le code généré par l'IA avant de l'intégrer dans l'application.

Ressources utilise :

- https://fr.wikipedia.org/wiki/Qualit%C3%A9_logicielle
- https://www.secondtalent.com/resources/ai-generated-code-quality-metrics-and-statistics-for-2026/
- https://www.coderabbit.ai/blog/why-2025-was-the-year-the-internet-kept-breaking-studies-show-increased-incidents-due-to-ai